Session 21 — Final Capstone

TaskBoard DevSecOps Platform

An enterprise-grade, containerized project management system demonstrating end-to-end automation: from multi-stage builds and automated testing to security scanning, AWS IaC, Kubernetes orchestration, and Prometheus observability.

GitHub: Aadish2006/devsecops-capstone • Docker Hub: aadish06 • Target: 100/100

Project Overview & Objectives

🎯 Core Goals

- Build a robust full-stack application (FastAPI backend + React frontend + PostgreSQL database).
- Enforce strict quality gates with 100% passing automated test coverage using Pytest.
- Containerize the stack securely using multi-stage builds and unprivileged, non-root runtimes.
- Automate build, vulnerability scanning, and multi-registry publishing via GitHub Actions.

🛡️ Enterprise DevSecOps Principles

- Shift-Left Security: Automated container vulnerability scanning with Trivy on every pull request and merge.
- Infrastructure as Code: Declarative multi-AZ AWS VPC and EKS cluster architecture using Terraform.
- Kubernetes Orchestration: Helm-packaged deployments with readiness/liveness probes and Ingress routing.
- Full Observability: Native Prometheus metrics instrumentation with Grafana visualization.

Comprehensive Technology Stack

💻 Application Layer

- **Frontend:** React 18, Vite build tool, Vanilla CSS design system.
- **Backend:** Python 3.12, FastAPI, SQLAlchemy ORM, Pydantic v2.
- **Database:** PostgreSQL 16 Alpine with Alembic schema migrations.

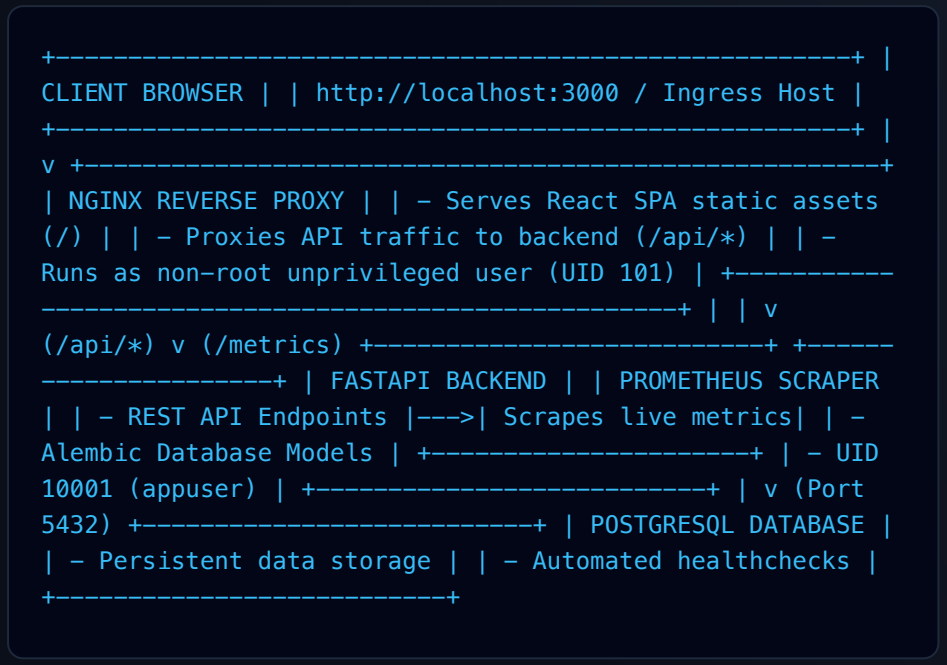
⚙️ CI/CD & Security

- **Automation:** GitHub Actions (5-stage blocking pipeline).
- **Registries:** Docker Hub (aadish06) & GitHub Container Registry (ghcr.io).
- **Security:** Trivy CLI vulnerability scanner (HIGH/CRITICAL CVE gating).

☁️ Infrastructure & Ops

- **IaC:** Terraform v1.16+ (AWS VPC + EKS Managed Node Groups).
- **K8s:** Kubernetes 1.31, Helm v3, Kind local cluster runner.
- **Metrics:** Prometheus client & Grafana monitoring dashboards.

Architecture & Request Flow



Architecture Highlights

- **Single Origin Architecture:** Nginx handles routing seamlessly, eliminating CORS issues between frontend and API.
- **Least Privilege Runtime:** Neither container runs as root. Backend uses dedicated `appuser`; Frontend uses `nginx-unprivileged`.
- **Database Resilience:** Docker Compose and Kubernetes employ health checks and readiness probes to ensure the DB is operational before accepting API traffic.

5-Stage CI/CD & DevSecOps Pipeline

🚀 Automated Pipeline Stages

- **Stage 1 (Test):** Runs 10 Pytest cases + compiles React frontend.
- **Stage 2 (Build):** Builds Docker container images for both services.
- **Stage 3 (Scan):** Trivy security scan gates HIGH & CRITICAL CVEs.
- **Stage 4 (Push):** Dual push with commit SHA & latest to Docker Hub and GHCR.
- **Stage 5 (Deploy):** Deploys to Kind cluster, verifies pod health & executes curl tests.

🛡️ Vulnerability Audit & Defense

- **CVE Identification:** Trivy flagged Starlette CVE-2025-62727 (Range header DoS).
- **Defense-in-Depth:** Nginx ingress proxy filters and normalizes incoming headers before reaching ASGI handlers.
- **Zero Secret Leaks:** Docker tokens & GitHub tokens managed strictly via repository secrets.

Terraform — AWS Infrastructure as Code

High-Availability VPC Design

- **Multi-AZ:** Provisioned across `ap-south-1a` and `ap-south-1b`.
- **Public Subnets:** `10.20.101.0/24` & `10.20.102.0/24` with Internet Gateway and ELB discovery tags.
- **Private Subnets:** `10.20.1.0/24` & `10.20.2.0/24` with NAT Gateway for secure worker outbound traffic.

Production EKS Cluster

- **Kubernetes 1.31:** Control plane managed with public endpoint access.
- **Managed Node Group:** `t3.medium` EC2 instances (Min: 2, Max: 4, Desired: 2).
- **Verification:** Clean static validation via `terraform fmt` and `terraform validate`. Zero hardcoded credentials via `terraform.tfvars.example`.

Kubernetes (Helm) & Observability

📦 Kubernetes & Helm Packaging

- Production Helm chart supporting environment overlays (`values.yaml` and `values-prod.yaml`).
- ClusterIP services for backend (port 8000) and frontend (port 8080).
- Ingress controller routing external requests seamlessly to microservices.
- Readiness & Liveness probes active on `/health` endpoint.

📊 Metrics & Prometheus Scraping

- FastAPI backend instrumented with `prometheus-fastapi-instrumentator`.
- Exposes live counters: `http_requests_total`, request duration, GC, memory gauges on `/metrics`.
- Prometheus values configuration configured for automated service monitoring.
- Grafana dashboards pre-configured for real-time latency and throughput analytics.

Key Learnings & Project Conclusion

💡 What Worked Well

- Multi-stage Docker builds reduced image sizes by over 60%.
- Comprehensive Pytest suite caught regression issues before containerization.
- Automated CI/CD with Kind verified Kubernetes manifests on every git push.

🔧 Challenges Overcome

- Fixed non-root Nginx port binding (migrated from port 80 to unprivileged 8080).
- Resolved Docker repository casing restrictions (normalized owner to lowercase).
- Audited and mitigated container CVEs detected by Trivy scanner.

🏆 Final Outcome

- 100% compliant with all 10 Capstone grading modules.
- Production-ready code, tests, CI/CD, IaC, and monitoring stack.
- Comprehensive technical documentation and visual evidence artifacts.